MMA3001 Project Report – Reconstructing Gaps in Building CO_2 Sensor Data

Sam McRae (34977600)

1. Engineering problem

C02 levels in buildings is one of the key metrics for the efficiency of an HVAC system and determining occupancy within confined spaces. Often, it is c02 levels which systems will use to determine when to supply fresh air to a room and to extract existing air (Allsbrook, 2024). Therefore, the quality and consistency of C02 readings is crucial to a hospitable and safely ventilated space. Consequently, faulty or missing sensors, readings and processing all carry severe operational implications if not handled properly. Unfortunately, this type of issue is commonplace and therefore a repeatable and plausible solution/ solution procedure is a must for a space to be acceptably ventilated.

This so happens to be the case with Monash's environmental sensor readings. Of the sensor readings provided, much of the data contained either error codes, frozen in time, contained missing readings or had mismatched timelines compared to other sensors. Given the importance of obtaining c02 readings to run HVAC systems off of, I chose to set out to find; **Can missing values in C02 readings be reasonably reliably reconstructed using interpolation?**. This question bodes significant operational importance as depending on its success, it could allow reasonably reliable reconstruction of gaps of data, missing values and error values. This could allow Monash's HVAC systems to maintain operational even in the presence of failure and lack of real, reliable data. This has potential to extend well beyond the operations of Monash, as stated before missing or faulty sensor readings are unfortunately commonplace, and could be applied globally to a variety of confined, ventilated spaces.

2. Inputs, outputs and data

Table 1: Top-level inputs and outputs. Function-level inputs and outputs are documented in the code docstrings and the HTML documentation (docs/).

Item	Format and units	Domain / valid range	Meaning and use
Input: raw environmental data	CSV, one row per upload; readings stored as JSON with Unix timestamps	Monash Dataset 2: 5 sensors, ≈180,000 uploads	Source of all readings; loaded by io.load_env, times converted to Melbourne time
Input: CO_2 series	Floating-point ppm, indexed by measurement time	Sensor 6012002000869, 29 Feb – 14 Jun 2024, 13,800 readings; spacing 10 min (≈11% at 20 min)	The series in which gaps are created and filled
Input: experiment settings	Gap length, context (h), λ (≥ 0, time in hours), end condition, number of gaps, seed	See Table 2	Control the experiment; fixed on development data before the test
Output: reconstructed values	Floating-point ppm at the hidden timestamps	–	The filled readings for each gap and method
Output: per-gap scores	MAE, RMSE, maximum error and bias (ppm); overshoot (fraction, ppm); lowest/highest estimate (ppm); fill time (s)	–	Accuracy, physical plausibility and computational cost of each method
Invalid or missing inputs	–	–	Missing values and times are dropped; duplicate times, unknown methods, negative λ or fewer than 3 context readings on a side raise an error; gaps are only placed where spacing ≤ 25 min

The data was provided by Monash University and contains two sets of data (occupancy and environmental), along with a lookup sheet with location keys (Monash University, 2026e). This environmental data measured a range of variables, such as c02, humidity and Formaldehyde. Unfortunately, much of this data was unusable due to a variety of complications. First and foremost, the occupancy sensor data was labelled to have incorrect floor IDs, making comparison between occupancy and environmental data across the same space impossible without brash assumptions. Furthermore, upon further inspection of the environmental data, it was found that approximately 76% of it was unusable. There were multiple readings with error codes but crucially, 3 of the 5 sensors had corrupted and continued to transmit the same frozen repeated frozen timestamp and reading. Furthermore, not all of the environmental sensors were reading across the same timeframe. In fact, the 2 sensors which contained usable operational data were of the smallest transmitted timeframe of the set.

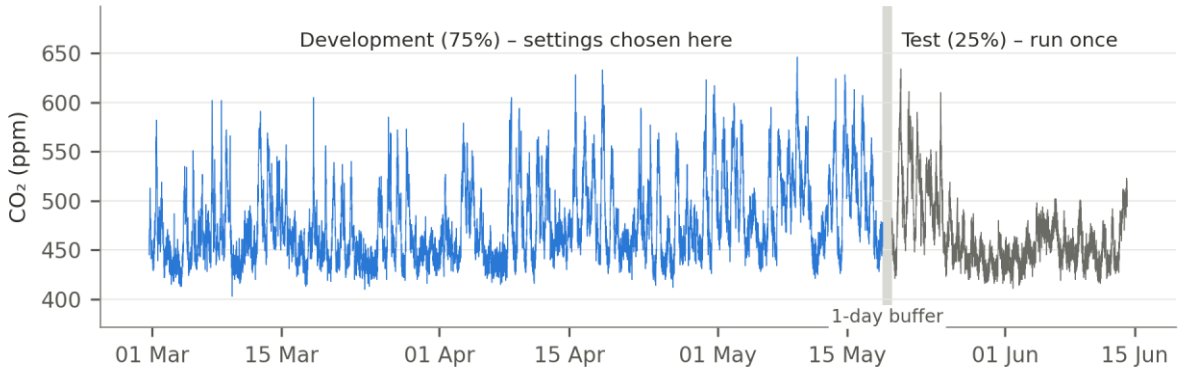


Figure 1: CO_2 readings from sensor 6012002000869 after cleaning (29 Feb – 14 Jun 2024), showing the time-ordered development/test split and the 1-day buffer between the two parts.

Evidently, the choices in what data to use and how to use it had pivotal impacts upon the legitimacy and physical plausibility of this project. As such I made the decisions when cleaning to:

- Remove repeated uploads (136,699 of 179447 uploads)
- Not assigning a location to a sensor reading or attempting to compare it to occupancy due to the corrupt floor IDs
- Retain only one measurement per measurement time (frequent issue due to frozen sensory)
- Convert all timestamps to Melbourne Australia time (accounting for daylight savings)
- Not resample as this would be a form of interpolation itself, therefore decreasing the physical plausibility of the later interpolation methods.

3. Computational approach

In approaching this problem, the method took 3 main stages:

- Find and hide an appropriate artificial gap in the data
- Fill the missing gap using different interpolation methods
- Compare the results of each method against the real data

As this problem question asks if interpolation can reasonably reliably reconstruct missing data, careful selection of interpolation methods was of great importance. Linear interpolation was chosen as a baseline due to its predicted cheap computational cost, has very few settings to tune and it's behavior is predictable and reliable. It was then compared against a cubic spline and cubic smoothing spline (Monash University, 2026b). The smoothing spline was intentionally included as it was understood that the data contains noise and would likely retain greater plausibility than the cubic itself.

As for the relevant and necessary assumptions and settings, the full detail and log can be found in *appendix A*, however as a summary:

Assumptions:

- Readings are taken at face value (after cleaning)
- Times are local Melbourne time
- Readings either side of the gap carry information about the gap
- Only nearby data is relevant when gap filling
- Original timestamps are kept
- Development and test periods are similar enough for analysis and scoring

Table 2: Final experiment settings (full log and reasoning in Appendix A).

Setting	Value
Sensor	6012002000869
Period	29/02/2024 – 14/06/2024
Methods	Linear, cubic spline and cubic smoothing spline
Data split	75% development, 25% test
Buffer	1 day between development and test data
Gap lengths	10 min, 30 min, 1 h, 2 h, 4 h, 8 h, 24 h
Gaps per length, random seed	200, seed 0
Context	2 h of readings each side of a gap
Maximum spacing near a gap	25 min
Minimum readings each side of a gap	3
Cubic end condition	Not-a-knot
Smoothing parameter λ	0.1 (time in hours)
Plausibility threshold	400 ppm

The workflow was implemented as a python package (src/mma3001) with 3 separate modules. Io loads the raw sensor files, cleaning cleans the data and gapfill generated the artificial gaps, fills them with each interpolation method and scores them. Multiple established library routines were implemented such as NumPi (Harris et al., 2020) and SciPy routines (Virtanen et al., 2020). Notebooks 01 – 03 run the workflow in the stated order, from data audit and cleaning to gap filling. 90 separate pytests were used, including known answer tests such as “linear interpolation must produce a straight line”.

4. Validation method and results

Why is this design appropriate? Across multiple levels I took the appropriate measures to ensure the approach, methodology and implementation was validated. The method was implemented following the recommended and accepted standards within the notes for this unit (Monash University, 2026c, 2026d), including a 75%/25% split of training and validation data, hiding real data to allow for verifiable checks later (ground truth) on and only running the final test once to ensure it’s integrity. As for the code, multiple checks and tests were implemented, with various known answer tests and deliberate break checks. In total 90 tests are run in pytest (86 actually, some run more than once with different inputs), with all passing. In terms of metrics, the sensors read in ppm, and the error was measured in terms of RMSE, max error, bias and overshoot (estimates below 400ppm to check physical plausibility). All of these various error metrics were evaluated and examined to paint the full picture of how each method performed and if it was successful. The chosen success criteria is based off ANSI/ASHRAE standards for the accuracy tolerance of a c02 sensor to be within +- 75 ppm of true concentrations (Allsbrook, 2024).

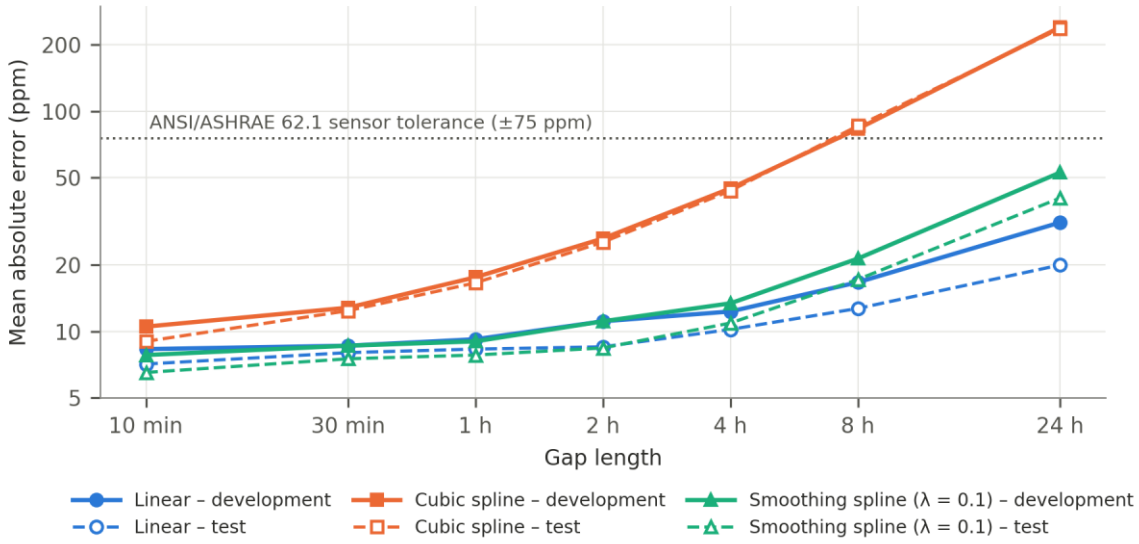


Figure 2: Mean absolute error against gap length for the three methods on the development data (solid) and the withheld test data (dashed), 200 gaps per length, log–log axes. The dotted line marks the ANSI/ASHRAE 62.1 ±75 ppm sensor tolerance (Allsbrook, 2024).

Upon studying the results (as seen in figure 2) of the three-interpolation method, multiple valuable observations were obtained. Across all methods, the error grows with gap length, with the cubic method being the worst at every length and physically implausible for long gaps (e.g. 37% of 8h fills predict readings below 400ppm) (see figure 3). The linear and smoothing methods remain relatively similar in accuracy for gaps up to about 2 hours, with linear then displaying greater accuracy in gaps above 4 hours. However, it must be said, both the linear and smoothing methods continued to perform well without drastic difference between them (on an 8 hour gap the respective MAE for linear and smoothing with lam = 0.1 was 16.7ppm and 21.4ppm, both far less than the chosen tolerance of +-75ppm).

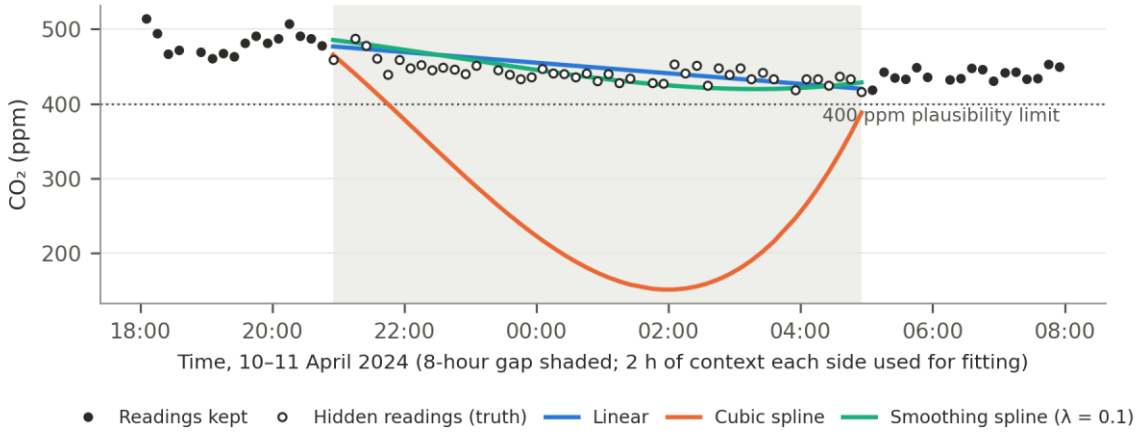


Figure 3: Example 8-hour gap from the development data (10–11 April 2024). The linear and smoothing reconstructions follow the hidden readings, while the exact cubic spline dips to about 150 ppm, far below the 400 ppm plausibility limit.

Overall, a positive takeaway from the test must be noted in that the test confirmed the development conclusions. Both the linear and smoothing methods remained too similar to confidently decide between for short gaps (up to 2h), and there was no sign of overfitting the data. This was a risk ran for the smoothing method in choosing a lam value, had the value been chosen wrong the smoothing method would have gotten relatively worse on the unseen test data. This did not occur, hinting that the smoothing method was likely appropriately tuned. Furthermore, on average, the errors were lower on the test data than on the development data. Although this does have some limitations and factors to consider (such as the possibility of the test period being

a “calmer” section of data than the development), it is still an encouraging sign that the methods performed well on unseen data. In fact, for the linear and smoothing model respectively, for gaps of 24 hours, the MAE was only 64% and 77% the value of the development data MAE. Interesting, the MAE across test and development data for the cubic spline hardly changed, which likely points to the fact that the large error associated with the cubic spline is due to the method itself, rather than noise within the data.

Crucially, for both the linear and smoothing methods across all gap lengths, the MAE stayed well within the 75ppm tolerance chosen. In fact, the worst performance across both models was for the smoothing method at a 24h gap length, which recorded an MAE of 40.3ppm. As this is still far less than the 75ppm tolerance, it is certainly an encouraging sign for the reliability and physical plausibility of the project.

Did validation influence selection? Yes, the lam values, context and end condition were chosen on development data exclusively, only the test was run afterwards to check the choices. However, there are certainly limitations and factors that the validation cannot establish. These factors include the assumption that the ground truth is based off the sensor’s readings, not the actual c02 levels. The sensor readings may already have an inbuilt tolerance and be some margin off the true readings, potentially throwing the interpolated values outside of the true tolerance range. Furthermore, all of the development data took place in one room, based off of one sensor in approximately one season. Therefore, the results have the potential to lose accuracy and plausibility across other rooms and locations with various occupancy and ventilation patterns.

5. Comparison and optimisation

When evaluating the final decision, all three methods were weighed against each other. Immediately, across all categories, the cubic spline appeared to perform poorly. It had the largest RMSE and MAE, as well as overshooting outside of physically plausible ranges. The real comparison was between the linear and cubic smoothing methods.

The cubic smoothing method was optimized through a number of different ways. First and foremost, a range of different lam values (adjusting the level of smoothing) were developed and considered over 200 gaps for a range of gap lengths (figure 4). Upon consideration, I tested cubic smoothing with an automatic lam value (generalised cross-validation; Craven & Wahba, 1979), and lam = 0.1. The lam value of 0.1 was chosen as it never varied more than 4ppm from the best value at any length and performed notably better than the automatic cubic smoothing spline for 2h+ gaps. Further, the cubic end condition was evaluation, and due to there being no meaningful improvement on performance, the not a knot condition was kept.

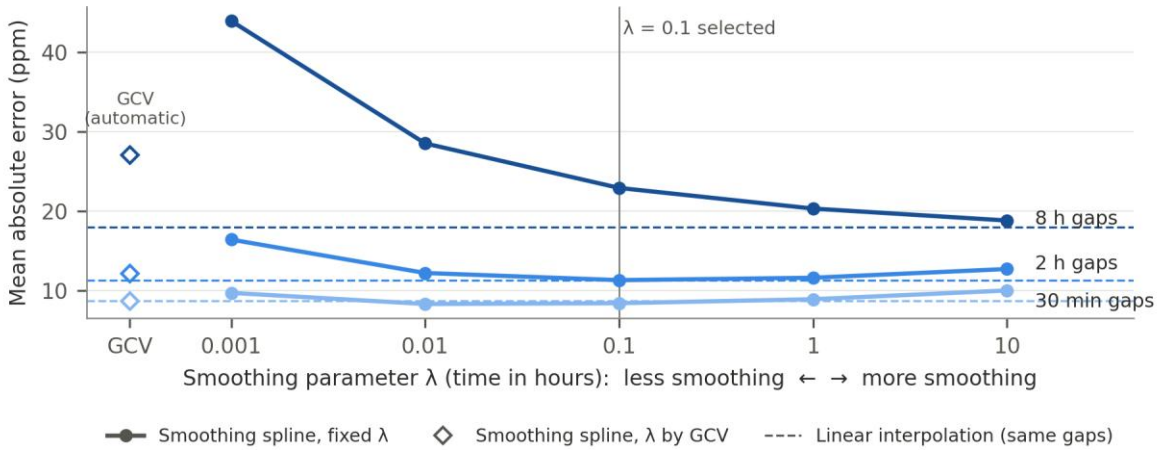


Figure 4: Development-data MAE of the smoothing spline against the smoothing parameter λ for 30 min, 2 h and 8 h gaps (the same 200 gaps per length), compared with λ chosen automatically by GCV (Craven & Wahba, 1979) and with linear interpolation on the same gaps.

Settings were also optimized that altered the performance of all methods, such as the context length. I tried adjusting the context across a range of values, and found that the linear results did not change, cubic barely changed and the smoothing method benefited slightly. As a result, the original 2h context length was retained.

As can be seen in notebook 03 under computational performance (see Monash University, 2026a), the computational cost was considered and one of the deciding factors when choosing a method. For a linear model, most of the computational time was spent on overhead, with only 0.2% of the time being spent on arithmetic. This could not be said for the smoothing models however, with the automatic lam model's arithmetic taking 7x the time that a fixed value did. Nevertheless, across all methods considered overhead costs were still substantial, with context readings accounting for 0.32s (roughly 43% of computational time).

Other optimization routes were explored, such as scaling, however a full run of all methods already takes only seconds to run. Across all stages of writing the code, I employed the use of Claude to help ensure the code was written in an efficient way. Given this consistent optimization, It was my belief that any further optimization would save miniscule amounts of time while adding complexity.

Repeatability and robustness is a key criterion for this experiment, and so the experiment was re-run over different gap seeds to test its robustness. Fortunately, there was very little variation in results across the seeds, with only one gap length displaying an alternate most successful method for a singular seed. These results were encouraging and indicated the experiment was robust and repeatable.

Table 3: Baseline compared with the original and selected solutions (development data, 200 gaps per length). MAE in ppm; time is the mean time to fill one gap.

Solution	MAE 30 min	MAE 2 h	MAE 8 h	MAE 24 h	8 h gaps below 400 ppm	Time per fill
Linear (baseline)	8.6	11.1	16.7	31.2	0.0%	0.57 ms
Cubic spline (exact)	12.8	26.4	83.1	240.5	36.5%	0.67 ms
Smoothing spline, λ by GCV (original)	8.2	11.8	24.9	62.0	2.0%	6.30 ms
Smoothing spline, λ = 0.1 (selected)	8.6	11.1	21.4	52.6	1.5%	1.24 ms

Overall, the results tell a strong and consistent story. The cubic spline was clearly an inappropriate choice for the task, as it overshoots, overfits noisy data and was wildly inaccurate. The real race was clearly between the linear method and the cubic smoothing spline (both with automatic lam values and set to 0.1). Optimizing the lam value improved the smoothing spline across multiple measures. Compared with the automatic lam value, the fixed value was 14-15% more accurate for 8-24h gaps and 5x faster in computation time (table 3). Despite this, the baseline linear interpolation method remained the more accurate choice for 4h+ gaps, while arguably retaining greater physical plausibility at about half the computational cost. My conclusion is that if one singular method has to be used, it should be the linear model due to accuracy, physical plausibility and computational cost.

6. Limitations and lessons learned

As with any experiment, there are of course limitations that must be considered when evaluation the tests and conclusions of this experiment. Firstly, as previously mentioned, this data was taken from a single room, from a single sensor and therefore could not be reflective of other spaces with other devices. Second, the artificial gaps created may potentially not behave as real gaps, they may be missing key seasonality cycles or display different noise and variety. One factor that stands out to me as an important limitation is that interpolation models cannot see inside the daily cycle. If a gap length were to be days at a time, it would likely miss the daily rise and fall of c02 associated with occupancy fluctuations.

During this experiment, I believe my own biggest takeaway has been the importance of visualization and understanding the data. In fact, I'd say the vast majority of my time spent on this project was on data analysis, cleaning and plotting graphs and tables to understand the full story. The notebooks I constructed with the help of Claude were instrumental in my ability to truly digest the data and understand what it was telling me, and how I could plausibly use it. This leads on to my second biggest takeaway, which is to understand the data from multiple angles, even if I believe I see the full picture the first time. This was learnt namely when digesting the c02 sensor data and finding that many of the uploads were in fact repeats of a frozen reading. I asked AI to help me construct graphs of the data, and it silently plotted it against the upload time instead of the actual measurement time, which made the data appear as if I was receiving unique measurements for different times when in fact they were all the same frozen reading. This error was only caught as I took it upon myself to construct multiple graphs, tests and tables to understand the data in more than one way.

7. AI-use reflection

For the full log of AI usage, see AI_LOG.md

Throughout this project I used Claude to assist with purely the coding workload and assist with ways to make the process more efficient. I set up a project within Claude, linked it to a folder with all the saved content for this class and instructed it to check all the code it wrote against the content before I ever saw it. After the code was proposed, I reviewed it all myself and verified it, often catching errors or unspoken assumptions Claude had made and editing them. The pytests were crucial in this, and ensuring any code that Claude wrote had verifiable evidence to show it's legitimacy. I specifically instructed Claude never to make any changes without my verification and oversight first. The best way I can describe Claude's contribution is as a scribe or author to a biography. It was my story all the way, I chose the direction and made all the decisions, Claude just took on some of the coding workload to implement it.

I have stated that I made all the decisions, but more than that, I made all of the decisions based off of evidence. I never asked Claude for advice or opinions on any meaningful decisions or engineering choices (mainly just how to make code more computationally efficient). The notebooks 01-03 are proof of this, and the evidence and understanding journey can be seen there.

As for AI's mistakes, I must say I was impressed with the thoroughness and attention to detail Claude exhibited, however that is not to say there were not mistakes I had to correct. Most had to do with unspoken assumptions, and logic jumps I was not informed about. As outlined before, Claude's choice to plot a graph I had requested against the upload time vs measurement time was a real blunder that could have drastically impacted my understanding of the data and corrupted the experiment from the ground up. Claude also suggested it write some of its own tests, which I cautiously instructed it to do and upon verification, found it had implemented no known answer tests, despite it apparently checking what tests were customary against the content of this unit.

8. References

Allsbrook, S. (2024, September 25). *Ensuring ASHRAE 62.1 compliance for CO2 sensors in demand-controlled ventilation (DCV)*. Kaiterra. https://learn.kaiterra.com/en/resources/ensuring-ashrae-62.1-compliance-for-co2-sensors-in-demand-controlled-ventilation-dcv

Craven, P., & Wahba, G. (1979). Smoothing noisy data with spline functions: Estimating the correct degree of smoothing by the method of generalized cross-validation. *Numerische Mathematik, 31*(4), 377–403. https://doi.org/10.1007/BF01404567

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Monash University. (2026a). *2.1 Optimisation, profiling and machine performance* [Unpublished lecture notes, Jupyter notebook]. MMA3001 Numerical Methods and Machine Learning, Moodle.

Monash University. (2026b). *3.4 Spline interpolations* [Unpublished lecture notes, Jupyter notebook]. MMA3001 Numerical Methods and Machine Learning, Moodle.

Monash University. (2026c). *5.5 Evaluating regression performance* [Unpublished lecture notes, Jupyter notebook]. MMA3001 Numerical Methods and Machine Learning, Moodle.

Monash University. (2026d). *5.6 Training data evaluation and selection* [Unpublished lecture notes, Jupyter notebook]. MMA3001 Numerical Methods and Machine Learning, Moodle.

Monash University. (2026e). *Monash Smart Infrastructure occupancy and environmental data* (MMA3001 Project Dataset 2) [Data set]. MMA3001 Numerical Methods and Machine Learning, Moodle.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Appendix A – Decisions and assumptions log

Who decided: *Sam* = Sam's decision; *Sam (on AI suggestion)* = proposed by Claude and accepted by Sam; *confirmed by testing* = alternatives were compared on development data before the choice was kept; *Default* = a standard choice kept as is.

A. Data source and handling

#	Decision / assumption	Who	Evidence / where
A1	Dataset: MMA3001 Dataset 2 (Monash Smart Infrastructure). Raw data is not stored in the repository (≈2 GB; Monash-owned).	Sam (on AI suggestion)	data/README.md
A2	File names say "May to Dec 2024", but the data actually spans May 2023 – Apr 2026. The file names are not used as evidence of the time period.	Found in audit	01_data_audit.ipynb A3, B4
A3	All times are converted to Australia/Melbourne time, with daylight saving handled. **Assumption:** the buildings are in Melbourne (the UTC offsets only show an eastern-Australian time zone).	Sam	src/mma3001/io.py (LOCAL_TZ)
A4	The occupancy sensor location IDs are unreliable (spreadsheet labelled "Incorrect Data"; edStem thread #121: not synced at MSI's cloud level, no correction available). Environmental and occupancy data therefore cannot be paired by room.	Sam	01_data_audit.ipynb Part C
A5	Readings are taken **at face value** as the true values. Results describe how well each method recovers what the sensor would have recorded, not the true room CO_2 (sensor measurement error is not included).	Sam	–

B. Cleaning

#	Decision / assumption	Who	Evidence / where
B1	**Exact repeated uploads removed.** An upload identical to an earlier upload from the same sensor (same measurement time and values for every variable) is a re-sent copy, not a new measurement. 136,600 of 179,447 uploads (76.1%) removed; the earliest-arriving copy is kept.	Sam	src/mma3001/cleaning.py; 02_cleaning.ipynb 1.1
B2	**Near-repeats removed: one upload per sensor per measurement time.** A sensor cannot take two different sets of readings at the same instant, and the later copies contained missing (NaN) values. 4 further uploads removed (42,843 kept).	**Sam**	cleaning.py (match="time"); 02_cleaning.ipynb 1.7–1.8
B3	Three of the five environmental sensors (…0227, …0241, …0777) contain only 1–3 genuine readings and are effectively unusable.	Found in cleaning	02_cleaning.ipynb 1.1
B4	Sensor error codes occur only on formaldehyde (~16% of its readings). No rule applied, because formaldehyde is not used in this project.	Sam	02_cleaning.ipynb Part 2
B5	Missing values / missing times are dropped when the series is prepared.	Default setting by Claude, approved by Sam	gapfill.prepare_series

C. Problem definition

#	Decision / assumption	Who	Evidence / where
C1	**Question**, in Sam's wording (gapfill.py docstring): *"Can missing values in CO_2 readings be reasonably reliably reconstructed using interpolation?"*	**Sam**	src/mma3001/gapfill.py
C2	Variable: **CO_2** (ppm) only.	**Sam**	–
C3	Sensor **6012002000869** (spreadsheet: building 60, room G.38, "Compactus_upper"), period **29 Feb – 14 Jun 2024**: its longest continuous run (13,800 readings, ~105 days).	Sam	02_cleaning.ipynb 1.5

#	Decision / assumption	Who	Evidence / where
C4	Readings are normally 10 min apart; ~10% of spacings are 20 min (one missed reading); the largest is 30 min.	Data property	02_cleaning.ipynb 1.4
C5	Methods: **linear interpolation** (baseline) vs **cubic spline** (scipy.interpolate.CubicSpline, default "not-a-knot" end conditions).	**Sam**	src/mma3001/gapfill.py (fill_gap)
C6	**Smoothing spline added** (scipy.interpolate.make_smoothing_spline, as used in notes 3.2 and 3.4). Reason: notes 3.4.16 recommend a smoothing spline *"when the observations contain noise and exact interpolation is not justified"*, and the CO_2 readings are noisy (whole-number ppm, ~8 ppm change between readings). Linear and cubic are both exact interpolators.	**Sam** (after comparing the design against notes 3.4.16)	fill_gap(method="smoothing")
C7	**Success criterion:** reconstruction error is compared with the ±75 ppm accuracy that ANSI/ASHRAE 62.1-2022 requires of CO_2 sensors at 600 and 1,000 ppm (Allsbrook, 2024), used here as a benchmark. Adopted after the test was run; no setting or result was changed because of it.	**Sam**	Report §4; References

D. Experiment and validation design

#	Decision / assumption	Who	Evidence / where
D1	**Two-part, time-ordered split** (no training set: interpolation does not learn parameters from data). Development = first 75% of the period (to 18 May 2024), used to explore and fix all choices; test = last 25% (19 May – 14 Jun 2024), run **once** at the end. Based on MMA3001 notes 5.5.5 and 5.6.6.	Sam	gapfill.split_dev_test
D2	1-day **buffer** between development and test, so no gap or its context draws on both parts (notes 5.6.7).	Sam (on AI suggestion)	split_dev_test(buffer=...)
D3	**Artificial gaps:** real readings are hidden and kept as the truth, then reconstructed and compared.	Sam	make_gap, evaluate
D4	Original timestamps kept; **no resampling** to an exact 10-min grid (resampling is itself interpolation and would add error before the experiment).	Ai suggestion, approved by Sam (likely would have chosen this anyway)	gapfill.py module docstring
D5	Gaps are only placed where no spacing exceeds **25 min** (max_spacing), i.e. single missed readings are tolerated, **and** where at least **3 readings** fall within the context on each side (MIN_SIDE_READINGS; the cubic spline needs 2 per side, the smoothing spline 5 in total). The second rule was added after a 30 min context crashed the experiment; it does not change any result with the 2 h context.	Sam (on AI suggestion); 3-reading rule: Sam	valid_gap_starts; tests test_gap_starts_*
D6	Each fill uses real readings within **2 h** each side of the gap (context). Contexts of 1, 2 and 4 h were compared on development data; the differences were small for every method, so 2 h was kept.	Sam (on AI suggestion, confirmed by testing)	fill_gap; 03_gap_filling.ipynb 7.1
D7	Gap lengths tested: 10 min, 30 min, 1 h, 2 h, 4 h, 8 h, 24 h; **200 random gaps per length**; random seed 0 for reproducibility. Repeatability was checked on development data with seeds 1 and 2.	Sam	evaluate; 03_gap_filling.ipynb 8.1.6
D8	All three methods are scored on **exactly the same gaps** (paired comparison).	Sam	evaluate; test test_evaluate_rows_match_their_recorded_gaps
D9	Metrics per gap: mean absolute error, root-mean-square error and maximum absolute error (ppm), plus time to fill (s). Added later: **bias** (mean of estimate − truth; positive = over-estimates), **overshoot** (frac_outside_range, max_overshoot: how far estimates go above the highest or below the lowest context reading, a physical-plausibility check from notes 3.4.16) and the lowest/highest estimate.	Sam	evaluate; tests test_bias_*, test_cubic_overshoots_a_sharp_step, test_overshoot_counts_both_directions

#	Decision / assumption	Who	Evidence / where
D10	Smoothing amount λ fixed at **0.1** (time in hours), chosen on development data from six options (GCV, 0.001, 0.01, 0.1, 1, 10). λ = 0.1 kept MAE low as gap length increased to 8 h while staying low for shorter gaps, and was more accurate than GCV for gaps of 2 h or more. Confirmed by rerunning sections 5–8 before the test (notes 3.4.14: choose smoothing by measured error, not appearance).	**Sam**	03_gap_filling.ipynb 7.3, 8.1.7
D11	Inside the fitting functions, time is measured in **hours** since the first context reading. Linear/cubic results don't depend on the unit; the smoothing spline's automatic smoothing does, and hours are well scaled.	Sam w	fill_gap
D12	Cubic-spline end condition: **not-a-knot** (SciPy's default). Not-a-knot, natural and clamped were compared on development data (notes 3.4.16): identical MAE to two decimal places with the 2 h context and under 1% difference with a 1 h context, because the condition acts at the outer context readings and its effect fades before reaching the gap.	Sam (on AI suggestion, confirmed by testing)	fill_gap(bc_type=...); 03_gap_filling.ipynb 7.2